

UNIVERSITY OF MORATUWA

DEPARTMENT OF ELECTRONIC AND TELECOMMUNICATION
ENGINEERING



Routing Protocol Design

Team ElectroSquad

ABEYWARNA D.H. 230013A
CHANDRAKUMARA H.A.D.C. 230100M
HANSINDU M.M.A.D. 230229P
WEERASINGHE J.A.H.R. 230697V

May 16, 2026

Contents

1	Introduction	2
2	Existing Protocols and Their Limitations	2
2.1	RIP	2
2.1.1	Limitations of RIP	2
2.2	OSPF	3
2.2.1	Limitations of OSPF	3
2.3	IS-IS	5
2.3.1	Limitations of IS-IS	5
2.4	BGP	6
2.4.1	Limitations of BGP	6
3	Protocol Design	7
3.1	Routing Metric	8
3.2	Control Message Format	8
3.2.1	Hello Packet	8
3.2.2	LSA Packet	9
3.3	State Information at Routers	10
3.4	Algorithm	10
3.4.1	Rapid Hellos and Fast Convergence	10
3.4.2	RVF verification	12
4	Performance Analysis Using Simulation	14
4.1	Simulation Methodology	14
4.2	Simulation Results	15
4.3	Protocol Validation Summary	17
4.4	Simulation Code - Drive Link	17
5	Security & Scalability Analysis	17
5.1	Overview of Protocol Enhancement	17
5.2	Scalability and Convergence Behavior	18
5.3	Resistance to Route Hijacking and Spoofing	18
5.4	Replay Attack and State Consistency	18
5.5	Loop Prevention and Path Stability	18
5.6	Failure Points and Limitations	18
5.7	Suggested Improvements	19
5.8	Conclusion	19
6	Conclusion	19
7	Contribution of Team Members	20

1 Introduction

Networks, especially the internet, depend on routing protocols to move data efficiently through it. These protocols determine how routers may communicate with each other, share information, and decide on the best path for data travel from a source to a destination. Since the beginning of the information age, routing protocols have been developed, RIP, OSPF, IS-IS and BGP being some of the examples. These protocols focus on solving specific problems, and form the backbone of the modern internet.

However, since these were designed at a time of relatively stable, simple, wired infrastructure with infrequent failures, for modern networks with dynamic, wireless links, real-time applications and security concerns, limitations of these protocols might become more apparent. Issues like slow failure detection and convergence, rigid metric systems, LSA falsification vulnerabilities, and scalability bottlenecks are documented, as well as affecting everyday network performance.

This report presents an analysis of the core limitations of RIP, OSPF, IS-IS and BGP, followed by the design of an improved routing protocol, OSPF-RA (OSPF – Rapid and Authenticated). This new protocol targets two specific weaknesses of OSPF, namely, slow convergence, and susceptibility to LSA falsification. It introduces a Rapid Hello mechanism with a Link Performance Score metric for faster and more intelligent failure detection, and a Router Verification Field for cryptographic LSA authentication. Finally, the report presents simulation results comparing OSPF-RA against standard OSPF, and evaluates the security and scalability properties of the proposed protocol design.

2 Existing Protocols and Their Limitations

2.1 RIP

RIP (Routing Information Protocol) is one of the earliest distance-vector routing protocols designed for communication within a single autonomous system [1]. Its operation is based on routers periodically exchanging routing tables with neighboring routers and selecting the route with the smallest hop count [1], [2]. Due to its simplicity, RIP became popular in small and early computer networks. However, modern networking environments expose several weaknesses in the protocol.

Unlike modern routing protocols that maintain a detailed map of the network, RIP routers only know the distance and next hop to a destination [2], [3]. This limited network awareness makes routing decisions less intelligent and less adaptive to dynamic conditions.

2.1.1 Limitations of RIP

1. Limited Network Size

A major drawback of RIP is its restricted network size. Since the protocol defines 16 hops as infinity, communication across large networks becomes impossible [1], [4]. This limitation prevents RIP from being practical in enterprise-scale infrastructures or highly distributed systems [5].

2. Slow Response to Failures

RIP relies on periodic updates instead of immediate topology synchronization. As a

result, routers may continue using invalid paths for a significant amount of time after a failure occurs [1], [3]. During this period, packets may circulate unnecessarily or become lost before the network stabilizes again [2].

3. Count-to-Infinity Problem

RIP suffers from the count-to-infinity problem, where routers repeatedly exchange outdated routing information after a link failure [1]. Although mechanisms such as split horizon and route poisoning attempt to reduce this issue, they cannot fully eliminate routing instability in larger topologies [4], [3].

4. Inefficient Path Selection

The routing metric used by RIP is another weakness. Decisions are based solely on hop count, while important network conditions such as bandwidth, congestion, delay, reliability, and link quality are completely ignored. Consequently, RIP may choose paths with fewer routers even if those routes provide significantly worse performance [2], [5].

5. High Routing Overhead

RIP continuously broadcasts entire routing tables at fixed intervals, regardless of whether the topology has changed [1], [4]. In networks with many routers, this consumes bandwidth unnecessarily and increases processing overhead [3].

6. Weak Security Mechanisms

Security is also a concern in RIP. Earlier versions of the protocol lacked authentication mechanisms, allowing malicious devices to inject false routing information [1]. Even with authentication support in RIPv2, compromised routers can still distribute incorrect routes and disrupt traffic flow [4], [5].

Overall, RIP was designed for small, stable wired networks where simplicity was prioritized over efficiency and scalability. While it remains useful for learning fundamental routing concepts, its limitations make it unsuitable for modern large-scale, high-speed, and dynamically changing networks [2], [3].

2.2 OSPF

OSPF is a link-state interior gateway protocol, and operates within a single autonomous system [6], (for example, a single organization's network). The core idea of OSPF is each router building a complete map of the network. To do this, first they will establish adjacencies with their neighbors by exchanging Hello packets, then generate Link State Advertisements (LSA) which describe the router's local connections and their costs. These LSAs are sent throughout the network, and every router will use these LSAs to create a Link State Database (LSDB), which is a copy of the whole network topology. Then, Dijkstra's algorithm will be used to find the best paths to each destination in the LSDB, and the results will be stored in the router's routing table.

OSPF's routing metric is the cost of the path. This cost is taken as inversely proportional to the link's bandwidth, so the more bandwidth a link has, the lower cost it will have.

2.2.1 Limitations of OSPF

1. Scalability

OSPF uses a hierarchical area structure to manage scale. There's a backbone area in the center (Area 0), and other areas are connected to it. There are Area Border Routers (ABRs) at the boundaries between areas, and they summarize routing information of the areas.

However, in a single-area OSPF, when the number of routers and links are high, the backbone area will be swamped with large amounts of routing information, causing slow convergence. Even in a multi-area network, lower-level areas cannot directly communicate with each other, and so all inter-area traffic is still forced to go through the backbone. This creates a bottleneck at Area 0 (backbone).

2. Convergence

When a link or router fails, OSPF has to detect that failure, rebuild the adjacencies, flood the network with LSAs, and recalculate routing tables before the network stabilizes again. All of these steps take time, and the network might be unaware of a failure for several seconds before convergence even begins. But we cannot reduce the detection time too much, because that will increase the number of false alarms. [7]

And with large-scale networks, each time something changes, routers will flood the network with LSAs, including duplicates. This will waste bandwidth and overload the CPUs. Not to mention, additional Hello packets will be missed, which will lead to more failures.

3. Security

A serious vulnerability in OSPF is LSA falsification. Since the LSA contains link information, an injected false LSA, if accepted by even one router, can alter the entire network's idea of the topology. Researchers have demonstrated this by using attacks such as 'Remote False Adjacency' and 'Disguised LSA'. Remote False Adjacency involves fooling a remote server to advertise a non-existing link, and Disguised LSA allows the attacker to completely control the entire LSA content of a remote router. [8]

Attacks like these can lead to Denial-of-Service (DoS) attacks, eavesdropping, or man-in-the-middle attacks. And while OSPF does support cryptographic authentication, it only protects the packets in transit, and would not prevent injecting false topology information from a compromised router.

4. Using only a single metric

OSPF uses only a cost (derived from bandwidth) as its routing metric. But in practice, even higher-bandwidth links can be congested, resulting in high latency, jitter and packet loss. There would be less congested, but lower-bandwidth links, but the protocol will ignore those.

5. Resource Intensive

Each time the topology changes, every router will store the full LSDB and run Dijkstra's algorithm. This is highly CPU and memory intensive, and can add very high processing overhead in networks with frequent topology changes.

In conclusion, this protocol, while prioritizing scalability, fast convergence, and efficient bandwidth usage, has several limitations that prevent it from being completely successful. OSPF, first introduced in 1989, was built with mostly static, wired infrastructure with infrequent failures in mind (Although OSPF was extended to support IPv6 in 2008 [9]). But in the modern world, with wireless communications, MANETs (Mobile Ad hoc Network), and real-time use cases, OSPF's limitations are significantly noticeable.

2.3 IS-IS

IS-IS (Intermediate System to Intermediate System) is a link-state interior gateway protocol that uses Dijkstra's algorithm to calculate the shortest path through a network. Every router running IS-IS builds a complete, identical map of the network topology by exchanging routing information. IS-IS uses Link State PDUs (LSPs) [10, 11].

IS-IS operates directly over Layer 2 (the data link layer). This means it does not actually need an IP address to form neighbor relationships; instead, it uses OSI network addresses [12, 13].

2.3.1 Limitations of IS-IS

1. Scalability and the Backbone Bottleneck

IS-IS manages scalability by breaking the network into a two-level hierarchy: Level 1 (L1) for local areas and Level 2 (L2) for the backbone. Though IS-IS generally handles massive networks slightly better than some other protocols (such as OSPF), it still runs into structural bottlenecks.

As the network grows, the L2 backbone can become incredibly bloated. Every L2 router has to store the topology data for the entire backbone. In a massively scaled single domain, a single topology change forces a flood of LSPs across the entire backbone, forcing every L2 router to recalculate its routing table, which can seriously bog down network performance [13, 11].

2. Convergence and Flapping Links

When a link goes down, IS-IS has to detect the failure, generate new LSPs, flood them across the network, and run the Shortest Path First (SPF) algorithm again. In ideal conditions this happens quickly, but balancing detection time is complex.

If engineers set the Hello timers too high, the network takes too long to detect failed routers, leading to dropped packets. If the timers are configured too aggressively to improve convergence speed, routers become hypersensitive. An unstable or "flapping" link can cause routers to continuously flood the network with LSPs and repeatedly trigger CPU-intensive SPF calculations, creating network-wide instability [12, 10].

3. Security Vulnerabilities

Because IS-IS runs at Layer 2, it is immune to many common IP-based remote attacks. Attackers generally need access to the same local network segment to interfere with the protocol. However, this does not make IS-IS completely secure.

If an attacker compromises a router or gains access to the local broadcast domain, spoofed LSPs can be injected into the network. IS-IS supports authentication mechanisms such as HMAC-MD5 to verify legitimate routers, but the protocol does not encrypt topology information inside packets. A compromised router can advertise false topology information, allowing attackers to reroute traffic, create black holes, or intercept sensitive data [13, 11].

4. Rigid Metric System

IS-IS was originally designed with a rigid metric system where the maximum link cost was limited to 63. As a result, the protocol could not properly differentiate between high-speed modern links and slower legacy links.

Although modern implementations introduced wide metrics to overcome the numbering limitation, the metric is still largely static and mainly bandwidth-based. IS-IS does not dynamically account for real-time network conditions such as congestion, jitter, latency, or packet loss. Consequently, traffic may continue to use suboptimal paths even when better alternatives exist [10, 12].

5. Complexity and Resource Overhead

Like other link-state protocols, IS-IS requires routers to maintain a complete network topology database in memory and repeatedly execute Dijkstra's SPF algorithm whenever topology changes occur. This creates significant CPU and memory overhead, especially in large-scale networks.

In addition, IS-IS is considered relatively difficult to configure and troubleshoot. Since it relies on OSI Network Entity Titles (NETs) instead of standard IP addressing for its internal routing operations, many engineers find it more complex compared to IP-native protocols [12, 13].

In conclusion, although IS-IS is highly scalable and widely used in large service provider networks, it still faces limitations related to convergence, scalability bottlenecks, security, metric flexibility, and operational complexity [13, 11].

2.4 BGP

BGP (Border Gateway Protocol) is a path-vector exterior gateway protocol used for communication between different autonomous systems (ASes) on the Internet. Unlike OSPF, which operates inside a single organization, BGP is mainly used between Internet Service Providers (ISPs), cloud providers, universities, and large organizations. Being a path-vector protocol means that routers advertise reachable networks together with the complete path required to reach them. This allows routers to make routing decisions while also preventing routing loops. [14]

Routers establish peer connections with neighboring BGP routers and exchange routing information using UPDATE messages. Each router then selects the best route based on attributes such as AS-path length, local preference, MED (Multi Exit Discriminator), and routing policies. Unlike many routing protocols that mainly focus on shortest paths, BGP is heavily policy-based. This allows organizations to control traffic flow according to business agreements, security policies, or traffic engineering requirements. [14]

2.4.1 Limitations of BGP

1. Security Vulnerabilities

One of the biggest weaknesses of BGP is its lack of strong built-in security. BGP routers generally trust route advertisements received from neighboring routers. Because of this, a router can accidentally or maliciously advertise false routing information, which can spread across the Internet very quickly.

A well-known real-world example occurred in 2008 when Pakistan Telecom accidentally advertised incorrect routes for YouTube in an attempt to block YouTube locally. Those incorrect routes propagated globally, causing YouTube to become inaccessible in many parts of the world. This type of attack is known as prefix hijacking. [15]

2. Slow Convergence

When a network failure occurs, BGP routers must exchange UPDATE and WITHDRAW messages, recalculate routes, and propagate changes throughout the Internet before the network becomes stable again. During convergence, packets may be delayed, dropped, or routed inefficiently. In large-scale networks, convergence may take several minutes. [14]

3. Scalability Challenges

As the Internet continuously grows, BGP routers must maintain extremely large routing tables containing hundreds of thousands of network prefixes. Storing and processing these routing tables requires large amounts of memory and CPU resources, making scalability a major challenge. [14]

4. Policy-Based Routing Complexity

BGP routing decisions depend heavily on routing policies rather than shortest paths. Organizations often choose routes based on business agreements instead of actual network performance. As a result, traffic may sometimes travel through longer or slower paths. [14]

5. Route Instability and Flapping

BGP is vulnerable to route instability, called route flapping, where routes repeatedly appear and disappear because of unstable links or configuration issues. Frequent route updates increase CPU usage and create instability across networks. [15]

6. Resource Intensive Operation

BGP routers continuously process routing updates, maintain peer sessions, apply routing policies, and manage extremely large routing tables. This creates significant CPU, memory, and bandwidth overhead, especially in large backbone networks. [14]

In conclusion, although BGP is the foundation of global Internet routing and provides excellent flexibility and scalability, it still suffers from major limitations related to security, convergence speed, routing complexity, and resource usage. [15, 14]

3 Protocol Design

For our protocol design (OSPF-RA, Rapid and Authenticated), we decided on improving upon the current limitations of OSPF, particularly in areas of convergence, and security. As mentioned in the previous section, OSPF uses a single metric for routing, and takes a comparatively longer time to detect failures. Also, OSPF authentication only protects packets in transit, and doesn't prevent an already compromised router from injecting false information. To address these problems, the following improvements are proposed:

1. Add a second metric based on Hello response times and Hello miss rates (To diagnose the cost of links more accurately than just from bandwidth). Based on this metric, routers will send lightweight Hello packets more frequently (rapid mode) when the link condition is unstable and revert to normal frequency when link conditions are normal. This helps detect failures faster, while not unnecessarily flooding the network when it's stable.
2. A hash-based signature (let's call it Router Verification Field, or RVF), will be generated for each router, using the router's ID and a shared domain key. This signature will be attached to every LSA, and routers will reject LSAs whose signature doesn't validate.

3.1 Routing Metric

The existing cost metric will be kept but will be combined with the new metric (let's call it Link Performance Score, or LPS). LPS will be calculated from recent Hello response times and Hello miss rates over a sliding window, indicating latency and stability respectively. It will be a value between 0 and 1, and for path calculation, the effective cost will be,

$$\text{Effective Cost} = \frac{\text{Base Cost}}{\text{LPS}}$$

For calculating LPS, the following method will be used.

1. **Hello Success Rate** - What fraction of Hellos got a response out of a sliding window of the last N Hellos.

$$\text{Hello Success Rate} = \frac{\text{Successful Responses}}{\text{Total number of Hellos sent (over the sliding window)}}$$

2. **Latency Score** - The round trip time (RTT) of each Hello will be compared to a reference value (faster than normal RTT won't be considered)

$$\text{Latency Score} = \min(1, \frac{\text{Reference RTT}}{\text{Current RTT}})$$

From these values, LPS is calculated:

$$\text{Link Performance Score} = (0.7 * \text{Hello Success Rate}) + (0.3 * \text{Latency Score})$$

We give more weight to the Hello success rate because missing a Hello is a stronger signal than just being slow.

3.2 Control Message Format

3.2.1 Hello Packet

Table 1: Standard OSPF Hello Packet

Field	Size	Description
Network Mask	32 bits	Subnet mask of the sending interface
Hello Interval	16 bits	How often hellos are sent (default 10 seconds)
Options	8 bits	Optional capabilities flags
Router Priority	8 bits	Used for Designated Router election
Dead Interval	32 bits	How long before a neighbor is declared dead (default 40 seconds)
Designated Router	32 bits	IP address of the elected DR on this segment
Backup Designated Router	32 bits	IP address of the backup DR
Neighbor List	Variable	List of router IDs from which hellos have been received

Table 2: OSPF Common Header

Version	8 bits	OSPF version (2 for IPv4, 3 for IPv6)
Type	8 bits	Message type (1=Hello, 2=DBD, 3=LSR, 4=LSU, 5=LSAck)
Packet Length	16 bits	Total length of the packet
Router ID	32 bits	Unique ID of the sending router
Area ID	32 bits	Area this packet belongs to
Checksum	16 bits	Standard IP checksum
Authentication Type	16 bits	0=none, 1=simple password, 2=MD5
Authentication Data	64 bits	Credential data if authentication is enabled

Table 3: Added Fields

Hello Type	1 bit	0 = Normal, 1 = Rapid
Link Performance Score	8 bits	Current LPS value (0–255, normalized to 0–1)
Miss Counter	8 bits	Consecutive missed hellos on this link

The current Dead Interval for hello packets is fixed. Our protocol will override this value depending on stability, and switch to shorter intervals when instability is detected.

3.2.2 LSA Packet

Table 4: LSA Header

Field	Size	Description
LS Age	16 bits	Time in seconds since the LSA was originated
Options	8 bits	Optional capabilities
LS Type	8 bits	Type of LSA (1=Router, 2=Network, 3=Summary, etc.)
Link State ID	32 bits	Identifies what the LSA describes
Advertising Router	32 bits	Router ID of whoever generated this LSA
LS Sequence Number	32 bits	Incremented each time LSA is updated — used to detect newer versions
LS Checksum	16 bits	Integrity check of LSA contents
Length	16 bits	Total LSA length

Table 5: Type 1 Router LSA Body

Flags	8 bits	Whether router is ABR, ASBR, etc.
Number of Links	16 bits	How many links this router is describing
Link ID	32 bits	Identifies the neighbor or network
Link Data	32 bits	Interface IP address or mask
Link Type	8 bits	Point-to-point, transit, stub, etc.
Number of TOS	8 bits	Type of service entries (usually 0)
Metric	16 bits	Cost of this link

Table 6: Added Fields

Router Verification Field (RVF)	32 bits	HMAC-SHA256 truncated hash of LSA content + Router ID + Domain Key
RVF Timestamp	32 bits	Unix timestamp of LSA generation, prevents replay attacks

While there is a checksum field in original OSPF, it only checks if the packet was accidentally corrupted on the way. For detecting malicious modifications and when the packet was generated, these newly added fields would be useful.

3.3 State Information at Routers

- **Standard LSDB** – the original database from OSPF
- **Neighbor Health Table** – Stores current LPS, miss counter, current hello interval, and last hello timestamp per neighbor
- **Domain key** – Configured at deployment, and used for RVF generation and verification
- **RVF Cache** – Recent RVF values per router ID (used to detect replay attacks – resending valid data to gain malicious access or duplicate transactions)

3.4 Algorithm

3.4.1 Rapid Hellos and Fast Convergence

First, normal Hello packets will be sent to neighbors. If there is no response, after a certain threshold, the router will change the link's LPS value and switch into rapid mode, sending rapid Hello messages to that neighbor with shorter intervals. If the neighbor responds before the maximum number of rapid Hellos are sent, the connection is reestablished, but if the neighbor fails to respond in that time, the neighbor is declared as dead, and updated LSAs will flood the network.

Pseudocode

```

every hello_interval seconds:
    send Hello(type=NORMAL, LPS, miss_counter) to all neighbors

for each neighbor n:
    if no response received:
        n.miss_counter += 1

```

```

n.LPS = update_LPS(n)

if n.miss_counter >= RAPID_THRESHOLD:
    switch_to_rapid_mode(n)
else:
    n.miss_counter = 0
    n.LPS = update_LPS(n)

function switch_to_rapid_mode(neighbor n):
    set hello_interval = RAPID INTERVAL      # In this case, 1 second
    rapid_success_count = 0

    repeat up to MAX_RAPID_ATTEMPTS (5) times:
        send Hello(type=RAPID, LPS, miss_counter) to n

        if response received:
            rapid_success_count += 1
            if rapid_success_count >= RECOVERY_THRESHOLD (5):
                return to NORMAL mode
            return
        else:
            n.miss_counter += 1

    declare n as DEAD
    generate and flood updated LSA

```

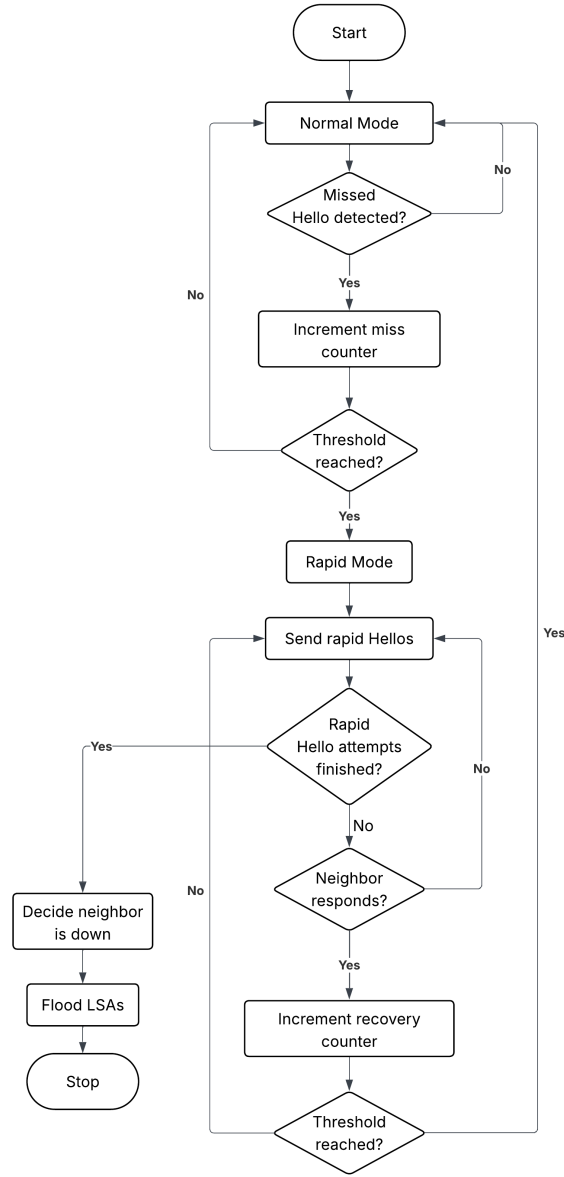


Figure 1: Flow Chart of Rapid Hellos and Fast Convergence

3.4.2 RVF verification

First, RVF (Router Verification Field) will be generated using the router's ID and the domain key. At the receiver, the LSA will be first checked for its timestamp, and if it's older than a max value we set (in this case, 60 seconds), the packet is discarded, since the LSA will have reached the receiver under normal circumstances by that time (This is to minimize replay attacks). If the timestamp checks out, then the expected RVF value is compared against the received RVF, and if they are not equal, the packet is discarded. If they do verify, the data in that LSA is used to populate the LSDB, and a modified Shortest Path First (SPF) calculation is run using the effective cost mentioned above.

Pseudocode

```

function generate_RVF(LSA, router_id, domain_key):
    timestamp = current_unix_time()
    payload = LSA.content + router_id + timestamp
  
```

```

    rvf = HMAC_SHA256(payload, domain_key)
    return rvf, timestamp

function verify_RVF(LSA, domain_key):
    if LSA.timestamp is older than MAX_AGE (60 seconds):
        log "Replay attack"
        return FALSE

    expected_rvf = HMAC_SHA256(LSA.content + LSA.router_id + LSA.timestamp, domain_k

    if LSA.rvf != expected_rvf:
        log "RVF verification failed."
        return FALSE

    return TRUE

function process_LSA(LSA):
    if not verify_RVF(LSA, domain_key):
        discard LSA
        return

    update LSDB with LSA
    run modified_SPF()

function modified_SPF():
    for each link L in LSDB:
        L.effective_cost = L.base_cost / L.neighbor.LPS

    run Dijkstra(using effective_cost)
    update routing table

```

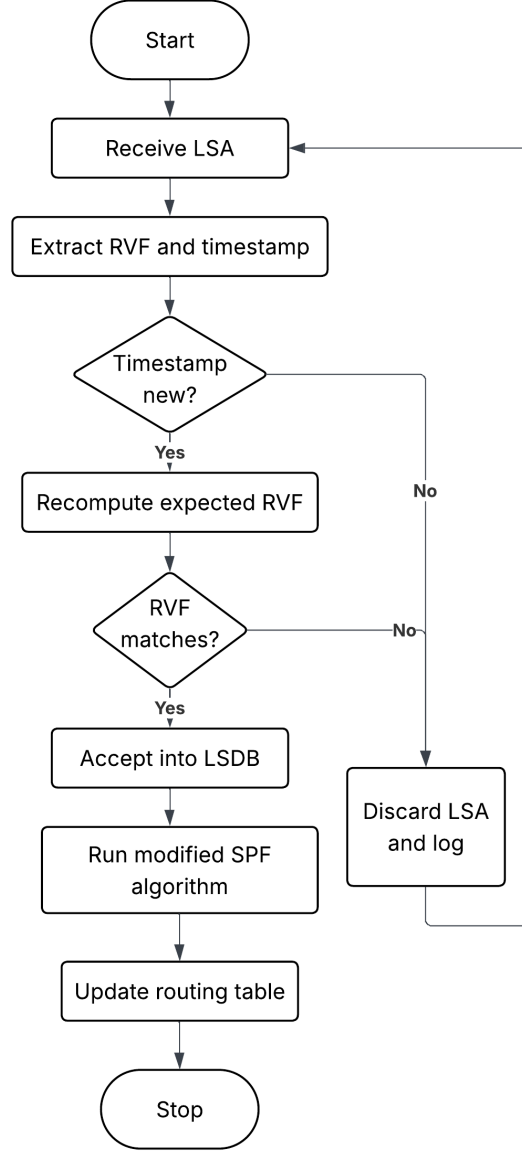


Figure 2: Flow Chart of RVF Verification

4 Performance Analysis Using Simulation

4.1 Simulation Methodology

To validate the proposed OSPF-RA protocol against standard OSPF, a discrete-event network simulation was constructed using Python and the `networkx` library. The environment models a 6-node network (R_1 through R_6) mapped with variable bandwidths and base link costs. The simulation executes side-by-side evaluations of four protocol enhancements:

1. **Rapid Failure Detection:** Tracks dropped Hello packets and triggers a high-frequency "Rapid Mode" (1-second intervals) upon missing a threshold, bypassing OSPF's rigid Dead Intervals.
2. **Dynamic Routing Engine:** Computes a continuous Link Performance Score (LPS) utilizing a sliding window of recent Hello success rates and latency metrics.
3. **Cryptographic Integrity:** Employs an HMAC-SHA256 hashing engine coupled with a

shared domain key to generate a Router Verification Field (RVF) for all Link State Advertisements (LSAs).

4. **Anti-Replay Architecture:** Enforces a strict 60-second Unix timestamp validity window and a localized RVF cache to neutralize duplicate or intercepted control messages.

4.2 Simulation Results

- **Convergence & Detection:** When an active transit node (R_3) fails, standard OSPF sustains a massive outage window due to its fixed 40-second Dead Interval. The proposed OSPF-RA mechanism detects the failure and shifts traffic significantly faster, relying on a maximum of 5 rapid polling attempts before declaring the link dead and initiating LSA flooding.
- **Proactive Traffic Engineering (LPS):** During simulated link degradation, the calculated LPS dynamically drops. As the LPS degrades, the effective operational cost ($Cost_{\text{eff}} = \frac{\text{Base Cost}}{\text{LPS}}$) inflates proportionally. This mathematical shift forces the routing algorithm to proactively calculate and utilize an alternative path ($R_1 \rightarrow R_4 \rightarrow R_5 \rightarrow R_6$) prior to absolute physical failure. Standard OSPF blindly remains routing packets into the failing path until total collapse.
- **Security & RVF Defense:** The simulation tested four cryptographic injection vectors. While standard OSPF accepts unauthenticated LSA states, the RVF engine achieved a 100% rejection rate against malicious payloads. It successfully dropped tampered LSAs (due to HMAC mismatches), expired historical LSAs (exceeding the 60-second window limit), and fresh cached duplicate replays.

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: D:\Documents\python\networksimulaton.py =====
OSPF vs OSPF-RA Simulation
=====

[1] CONVERGENCE SIMULATION
-----
Router R3 fails at t = 50s

Standard OSPF:
Detection : 40s (full Dead Interval = 4xHello)
Total : 43.0s
Converged : t = 93.0s

OSPF-RA (Rapid Hello):
Phase 1 : 2 missed x 10s = 20s (normal mode)
Phase 2 : 5 rapid x 1s = 5s (rapid mode)
Detection : 25s
Total : 28.0s
Converged : t = 78.0s

Improvement : 15.0s faster (34.9% outage reduction)

[1b] State-machine trace -- FULL FAILURE path:
t+ 10s [NORMAL] Hello sent -- no response. miss_counter=1
t+ 20s [NORMAL] Hello sent -- no response. miss_counter=2
t+ 20s [NORMAL] miss_counter=2 >= RAPID_THRESHOLD=2 -> switching to RAPID mode
(interval=1s)
t+ 21s [RAPID] Attempt 1: no response. miss_counter increments
t+ 22s [RAPID] Attempt 2: no response. miss_counter increments
t+ 23s [RAPID] Attempt 3: no response. miss_counter increments
t+ 24s [RAPID] Attempt 4: no response. miss_counter increments
t+ 25s [RAPID] Attempt 5: no response. miss_counter increments
t+ 25s [RAPID] MAX_RAPID_ATTEMPTS=5 exhausted -- neighbour declared DEAD.
Flooding updated LSA.

[1c] State-machine trace -- RECOVERY path:
t+ 10s [NORMAL] Hello sent -- no response. miss_counter=1
t+ 20s [NORMAL] Hello sent -- no response. miss_counter=2
t+ 20s [NORMAL] miss_counter=2 >= RAPID_THRESHOLD=2 -> switching to RAPID mode
(interval=1s)
t+ 21s [RAPID] Attempt 1: no response. miss_counter increments
t+ 22s [RAPID] Attempt 2: neighbour RESPONDED. recovery_count=1/5
t+ 23s [RAPID] Attempt 3: neighbour RESPONDED. recovery_count=2/5
t+ 24s [RAPID] Attempt 4: neighbour RESPONDED. recovery_count=3/5
t+ 25s [RAPID] Attempt 5: neighbour RESPONDED. recovery_count=4/5
```

```

t+ 26s [ RAPID] Attempt 6: neighbour RESPONDED. recovery_count=5/5
t+ 26s [ RAPID] recovery_count=5 >= RECOVERY_THRESHOLD=5 -> returning to NORMAL
mode

[2] LINK PERFORMANCE SCORE ( window N=10)
-----
Time  Win SR  RTT(ms)  LPS  Eff Cost  Mode
-----
0s    1.000  10.3  0.9919  10.08  NORMAL
5s    1.000  10.3  0.9947  10.05  NORMAL
10s   1.000  9.4   0.9954  10.05  NORMAL
15s   1.000  10.1  0.9966  10.03  NORMAL
20s   1.000  11.1  0.9936  10.06  NORMAL
25s   0.900  18.7  0.8808  11.35  NORMAL
30s   0.900  24.1  0.8025  12.46  NORMAL
35s   0.600  32.1  0.5379  18.59  RAPID <- RAPID
40s   0.300  42.5  0.3007  33.26  RAPID <- RAPID
45s   0.300  44.6  0.2853  35.05  RAPID <- RAPID
50s   0.300  39.8  0.2832  35.31  RAPID <- RAPID
55s   0.300  42.4  0.2837  35.25  RAPID <- RAPID
59s   0.400  44.0  0.3527  28.35  RAPID <- RAPID

[3] ROUTING PATH COMPARISON (R1 -> R6)
-----
Before failure:
R1 -> R2 -> R3 -> R6 (base cost: 30)

After R3 failure -- Standard OSPF:
R1 -> R4 -> R5 -> R6 (base cost: 60)

After R3 failure -- OSPF-RA (effective cost):
Link R4-R5: LPS=0.6 -> eff cost = 33.33 (vs base 20)
R1 -> R4 -> R5 -> R6 (eff cost: 73.33)

[4] RVF SECURITY VERIFICATION ( 4 scenarios)
-----
[1] Legitimate LSA from R3
Result : ACCEPTED
Reason : RVF verified successfully

[2] Tampered LSA (wrong-key RVF)
Result : REJECTED
Reason : RVF mismatch -- LSA falsification detected

[3] Old replayed LSA (timestamp expired)
Result : REJECTED
Reason : Replay attack -- LSA age 120s > limit 60s

[4] Fresh replayed LSA (RVF cache hit)
Result : REJECTED

Reason : Replay attack -- duplicate RVF in cache (fresh timestamp but already
seen)

[5] GENERATING SEPARATED MULTI-WINDOW PLOTS ...
Performance Trends plot saved -> simulation_performance_trends.png
Security and Topology plot saved -> simulation_security_topology.png

Done.
=====

```

Figure 3: Output of the simulation

OSPF vs OSPF-RA: Simulation Performance Metrics

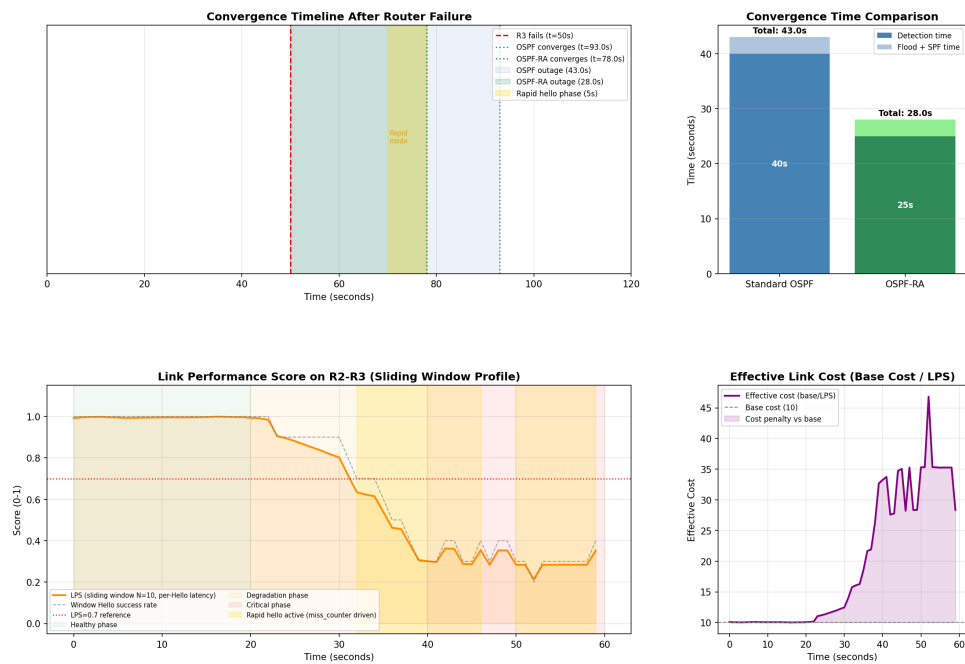


Figure 4: Timelines and Link performance composite metrics

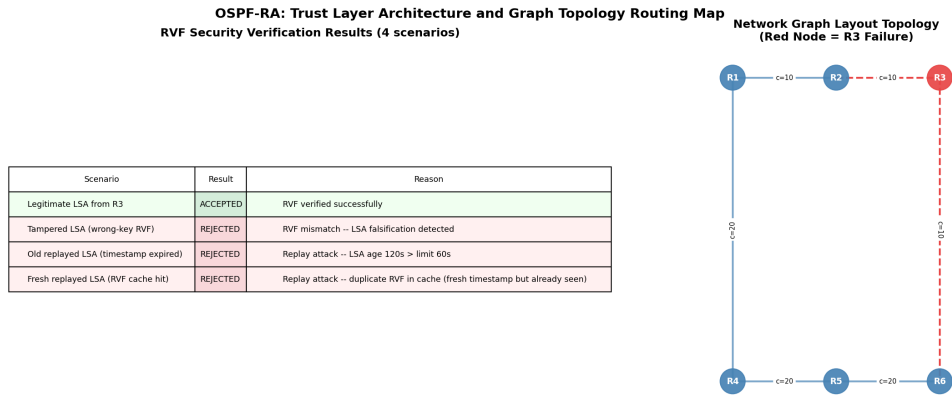


Figure 5: Security Matrix Representation Table and Connected Graph Map

4.3 Protocol Validation Summary

Standard OSPF consumed its full 40-second Dead Interval before initiating LSA flooding, converging at $t = 93s$ after the R3 failure. OSPF-RA's two-phase detection; two missed Normal Hellos triggering Rapid mode, followed by up to five one-second rapid attempts; brought convergence to $t = 78s$, a 34.9% reduction in outage duration. The state-machine trace confirms clean transitions between Normal and Rapid modes, with the recovery path correctly requiring five consecutive successful responses before reverting to normal Hello intervals.

During the healthy phase ($t = 0-20s$), LPS held near 1.0 and effective cost matched the base cost. As the R2-R3 link degraded through $t = 20-40s$, rising RTTs and increasing Hello miss rates drove LPS downward and inflated effective cost proportionally. By the critical phase, the effective cost on the primary path had risen sufficiently for the routing algorithm to prefer the alternate path $R1 \rightarrow R4 \rightarrow R5 \rightarrow R6$, with an effective cost of 33.33 against the degraded primary. Standard OSPF continued routing into the failing path until complete link disappearance from the LSDB.

All four adversarial scenarios were handled correctly. A legitimate LSA passed both the timestamp check and HMAC comparison cleanly. A tampered LSA signed with a wrong key was rejected on HMAC mismatch. An authentic but 120-second-old LSA was rejected by the freshness gate before HMAC evaluation. A fresh duplicate whose RVF value was already present in the local cache was rejected as a replay. The engine achieved a 100% rejection rate against malicious payloads with no false negatives under legitimate operation.

These results confirm that OSPF-RA meets its design goals, though future work should address dynamic domain key rotation and empirical tuning of the LPS window size and weighting parameters against real network traffic traces.

4.4 Simulation Code - Drive Link

[Link to Simulation code](#)

5 Security & Scalability Analysis

5.1 Overview of Protocol Enhancement

The proposed OSPF-RA extension introduces three major mechanisms: Rapid Hello-based failure detection, Link Performance Score (LPS)-driven adaptive routing, and cryptographic

Routing Verification Function (RVF) using HMAC-SHA256. Compared to standard OSPF, these enhancements target faster convergence, improved link-aware scalability, and stronger resistance against routing attacks such as spoofing, route hijacking, replay attacks, and topology inconsistencies.

5.2 Scalability and Convergence Behavior

Simulation results show a significant improvement in convergence performance under node failure (R3 failure at $t = 50s$). Standard OSPF relies on a fixed dead interval, resulting in a detection time of 40s and total convergence delay of 43s. In contrast, OSPF-RA reduces detection to 25s using adaptive rapid hello escalation, achieving full convergence in 28s.

This corresponds to a 34.9% reduction in outage time, demonstrating improved scalability in dynamic or large-scale networks where failure frequency and topology churn increase. However, scalability is partially limited by increased control-plane overhead due to frequent rapid hellos under degraded conditions, which may affect performance in very large topologies.

5.3 Resistance to Route Hijacking and Spoofing

The RVF mechanism provides strong cryptographic integrity for Link State Advertisements (LSAs). Simulation shows that legitimate LSAs are accepted only when HMAC verification, timestamp validation, and cache uniqueness checks all succeed. Spoofing attempts using incorrect keys are immediately rejected due to HMAC mismatch, while replayed LSAs are blocked via timestamp expiry and RVF cache detection.

These results indicate that OSPF-RA significantly improves resistance against route hijacking and identity spoofing attacks, as adversaries cannot inject or modify LSAs without domain key knowledge.

5.4 Replay Attack and State Consistency

Replay attack scenarios demonstrate that both stale and fresh replayed LSAs are successfully rejected. The combination of temporal validation and sliding-window RVF cache ensures that even recently valid LSAs cannot be reused maliciously. This prevents routing table poisoning and maintains state consistency across distributed routers.

5.5 Loop Prevention and Path Stability

Although OSPF inherently avoids routing loops through Dijkstra-based SPF computation, instability can occur during convergence delays. OSPF-RA improves stability by integrating LPS-based cost adjustment, which dynamically penalizes degraded links. As observed, the effective cost of unstable links increases significantly during degradation, discouraging their selection and indirectly reducing transient routing loops during reconvergence.

5.6 Failure Points and Limitations

Despite improvements, several limitations are observed. First, Rapid Hello mode introduces additional control traffic, which may reduce scalability in dense networks. Second, LPS depends on sliding-window history, which introduces short-term bias and may delay adaptation in rapidly fluctuating links. Third, RVF introduces computational overhead due to repeated HMAC computations, which may affect low-power routers.

5.7 Suggested Improvements

To enhance scalability further, adaptive hello intervals based on network density can be introduced to reduce unnecessary overhead. LPS computation can be improved using exponential weighted moving averages instead of fixed sliding windows for faster responsiveness. For security optimization, lightweight cryptographic primitives or hardware acceleration can reduce RVF processing cost while maintaining integrity guarantees.

5.8 Conclusion

Overall, OSPF-RA demonstrates improved scalability, faster convergence, and significantly stronger resistance to spoofing, route hijacking, and replay attacks compared to standard OSPF. While it introduces moderate computational and control overhead, the trade-off is justified in security-sensitive and dynamic network environments where fast failure recovery and routing integrity are critical.

6 Conclusion

This project analyzed the limitations of four widely deployed routing protocols and proposed OSPF-RA, a targeted enhancement to standard OSPF addressing convergence speed and LSA security. RIP's hop-count metric and 15-hop ceiling render it unsuitable for modern networks. OSPF improves on this with link-state routing but remains blind to real link conditions such as latency and packet loss, and its static Dead Interval causes slow failure detection. IS-IS shares these convergence trade-offs while adding operational complexity. BGP, designed for inter-domain routing, introduces convergence times extending to several minutes and an inherent trust model vulnerable to prefix hijacking.

OSPF-RA addresses these weaknesses through two enhancements. The Link Performance Score combines Hello success rates and round-trip latency over a sliding window into a composite routing metric, allowing the protocol to penalize degrading links proactively and reroute traffic before complete failure. The adaptive Rapid Hello mechanism automatically switches suspect links to one-second polling intervals upon detecting missed Hellos, then reverts to normal operation upon recovery. The Router Verification Field uses HMAC-SHA256 signatures bound to a shared domain key and a Unix timestamp, enabling routers to independently reject tampered, expired, and replayed LSAs. Simulation confirmed both enhancements. OSPF-RA reduced convergence time from 43 seconds to 28 seconds after a transit router failure, a 34.9% improvement. The LPS engine proactively rerouted traffic to an alternate path during simulated link degradation, well before total failure. The RVF engine rejected all four adversarial injection scenarios with a 100% success rate while correctly accepting legitimate LSAs. OSPF-RA demonstrates that meaningful improvements in convergence, adaptive routing, and security can be achieved within the existing OSPF framework, without requiring a complete protocol replacement.

7 Contribution of Team Members

Table 7: Team Member Contributions

Member	Assigned Role	Additional Contributions	Contribution (%)
WEERASINGHE J.A.H.R.	Literature Review	Assisted in protocol design and report writing	25%
ABEYWARNA D.H.	Protocol Design	Supported simulation implementation and literature review	25%
HANSINDU M.M.A.D.	Simulation	Assisted with protocol evaluation and debugging	25%
CHANDRAKUMARA H.A.D.C.	Security & Scalability Analyst	Supported protocol design and performance analysis	25%

References

- [1] C. Hedrick, “Routing information protocol,” RFC 1058, Internet Engineering Task Force (IETF), June 1988.
- [2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*. Hoboken, NJ, USA: Pearson, 8th ed., 2021.
- [3] J. Doyle and J. Carroll, *Routing TCP/IP, Volume I*. Indianapolis, IN, USA: Cisco Press, 2nd ed., 2005.
- [4] G. S. Malkin, “Rip version 2,” RFC 2453, Internet Engineering Task Force (IETF), November 1998.
- [5] Cisco Systems, “Configuring routing information protocol,” ip routing: rip configuration guide, cisco ios release 15m&t, Cisco Systems, San Jose, CA, USA, 2014.
- [6] J. Moy, “OSPF version 2,” RFC 2328, IETF, 1998.
- [7] M. Goyal, M. Soperi, E. Baccelli, G. Choudhury, A. Shaikh, and K. S. Trivedi, “Improving convergence speed and scalability in ospf: A survey,” *IEEE Communications Surveys & Tutorials*, vol. 14, no. 2, pp. 443–463, 2012.
- [8] G. Nakibly, A. Kirshon, D. Gonikman, and D. Boneh, “Persistent ospf attacks,” in *Proceedings of the Network and Distributed System Security (NDSS) Symposium*, pp. 1–12, February 2012.
- [9] R. Coltun, D. Ferguson, J. Moy, and A. Lindem, “OSPF for IPv6,” RFC 5340, IETF, 2008.
- [10] Wikipedia Contributors, “Is-is.” <https://en.wikipedia.org/wiki/IS-IS>, 2026. Accessed: 2026-05-16.
- [11] Huawei, “Is-is.” <https://info.support.huawei.com/info-finder/encyclopedia/en/IS-IS.html>, 2026. Accessed: 2026-05-16.
- [12] NetworkLessons, “Introduction to is-is.” <https://networklessons.com/is-is/introduction-to-is-is>, 2026. Accessed: 2026-05-16.
- [13] Cisco, “Intermediate system-to-intermediate system (is-is).” <https://www.cisco.com/site/us/en/products/networking/software/ios-nx-os/intermediate-system-to-intermediate-system-is-is/index.html>, 2026. Accessed: 2026-05-16.
- [14] Cloudflare, “What is bgp (border gateway protocol)?.” <https://www.cloudflare.com/en-gb/learning/security/glossary/what-is-bgp/>, 2026. Accessed: 2026-05-16.
- [15] Corero Network Security, “What is a border gateway protocol (bgp) attack?.” <https://www.corero.com/what-is-a-border-gateway-protocol-attack/>, 2026. Accessed: 2026-05-16.